

# STAT 8020 R Session 11: Random and Mixed Effects Models and Computer Experiments

Whitney Huang, Clemson University

## Contents

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Random Effects Example . . . . .                                            | 1  |
| Read the Data into R . . . . .                                              | 2  |
| Fitting a Fixed Effects Model . . . . .                                     | 3  |
| Fitting a Random Effects Model . . . . .                                    | 4  |
| Confidence Intervals for Variance Components and the Overall Mean . . . . . | 5  |
| RCBD: Fixed vs. Random Block . . . . .                                      | 5  |
| Load R Libraries . . . . .                                                  | 5  |
| Read the data . . . . .                                                     | 5  |
| Fixed Block Model . . . . .                                                 | 6  |
| Random Block Model . . . . .                                                | 7  |
| Pairwise Treatment Comparisons . . . . .                                    | 8  |
| Computer Experiments . . . . .                                              | 8  |
| Design: Latin Hypercube Design . . . . .                                    | 8  |
| Analysis: Gaussian Process Model . . . . .                                  | 10 |

## Random Effects Example

Suppose an agronomist is interested in soybean yield across a large population of soybean varieties. Rather than studying all possible varieties, the agronomist randomly selects three varieties and assigns each selected variety to 10 of the 30 available plots.

Because the varieties are randomly selected from a larger population of soybean varieties, variety should be treated as a **random effect** rather than a fixed effect.

The one-way random effects model is

$$y_{ij} = \mu + \alpha_i + \epsilon_{ij},$$

where

$$\alpha_i \stackrel{i.i.d.}{\sim} N(0, \sigma_\alpha^2), \quad \epsilon_{ij} \stackrel{i.i.d.}{\sim} N(0, \sigma^2),$$

and the random variety effects  $\alpha_i$  are independent of the plot-level errors  $\epsilon_{ij}$ .

Here:

- $y_{ij}$  is the yield from plot  $j$  receiving variety  $i$ ,
- $\mu$  is the overall mean yield,
- $\alpha_i$  is the random effect of variety  $i$ ,
- $\epsilon_{ij}$  is the plot-to-plot random error.

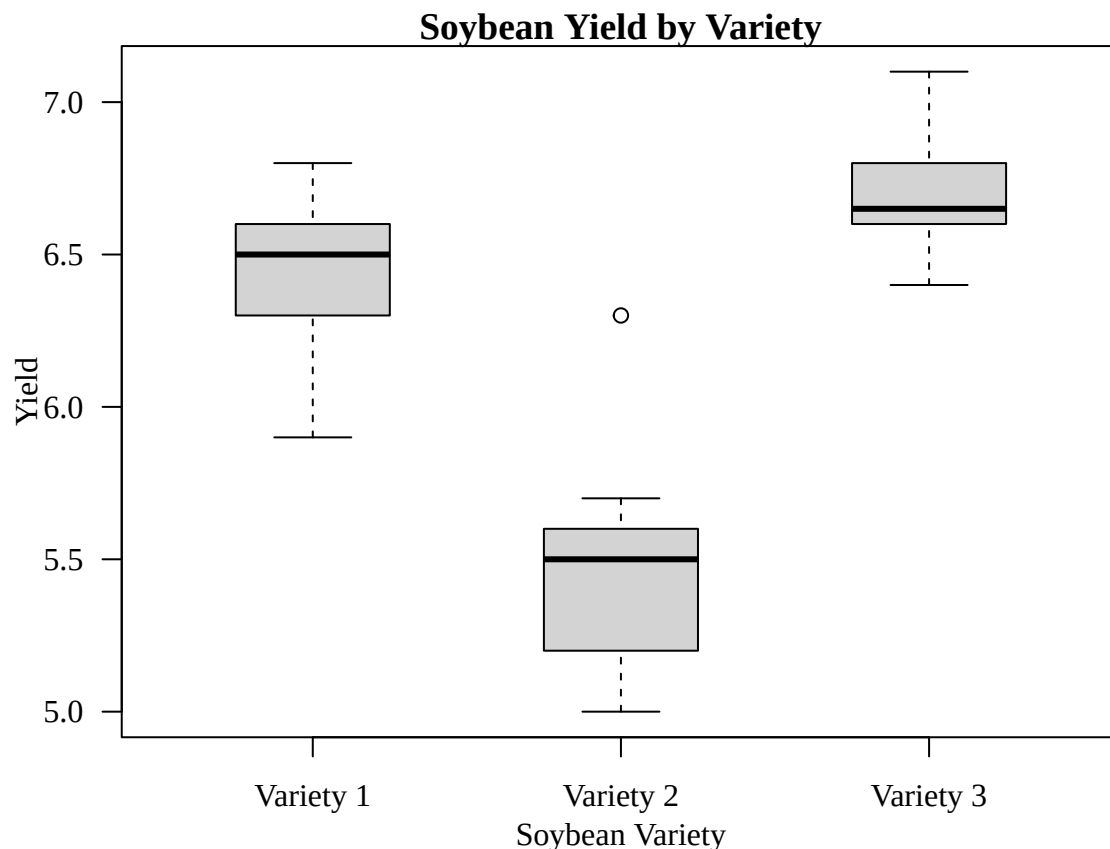
## Read the Data into R

```
# Yield data for three randomly selected soybean varieties
v1 <- c(6.6, 6.4, 5.9, 6.6, 6.2, 6.7, 6.3, 6.5, 6.5, 6.8)
v2 <- c(5.6, 5.2, 5.3, 5.1, 5.7, 5.6, 5.6, 6.3, 5.0, 5.4)
v3 <- c(6.9, 7.1, 6.4, 6.7, 6.5, 6.6, 6.6, 6.6, 6.8, 6.8)

# Combine the yield values into one response vector
yield <- c(v1, v2, v3)

# Create a factor variable identifying the soybean variety
variety <- factor(c(rep("Variety 1", 10),
                     rep("Variety 2", 10),
                     rep("Variety 3", 10)))

# Side-by-side boxplots to compare yield distributions across varieties
par(mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 1, 0),
    las = 1, family = "serif")
plot(yield ~ variety, boxwex = 0.5,
     xlab = "Soybean Variety",
     ylab = "Yield",
     main = "Soybean Yield by Variety")
```



The boxplot provides an initial visual comparison of the three selected varieties. It suggests that some of the variation in yield may be attributable to differences among varieties.

### Fitting a Fixed Effects Model

```
# Fit a one-way fixed effects ANOVA model
# This treats the three observed varieties as the only varieties of interest.
fixed_model <- lm(yield ~ variety)

# ANOVA table for testing whether the three variety means are equal
anova(fixed_model)
```

```
## Analysis of Variance Table
##
## Response: yield
##          Df Sum Sq Mean Sq F value    Pr(>F)
## variety    2  8.306  4.1530  49.593 9.114e-10 ***
## Residuals 27  2.261  0.0837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Estimated regression coefficients under R's default treatment coding
coef(fixed_model)
```

```
##      (Intercept) varietyVariety 2 varietyVariety 3
```

```
##                6.45                -0.97                0.25
```

In the fixed effects model, the varieties are treated as specific levels of interest. The ANOVA test asks whether the mean yields of these three particular varieties are equal.

This approach is appropriate if the goal is to compare only these three varieties. However, it is not ideal if the three varieties are viewed as a random sample from a larger population of soybean varieties.

## Fitting a Random Effects Model

```
library(lme4)
```

```
## Loading required package: Matrix
```

```
# Fit a one-way random effects model using restricted maximum likelihood  
# The term (1 | variety) allows each variety to have its own random intercept.  
random_model <- lmer(yield ~ 1 + (1 | variety), REML = TRUE)
```

```
# Model summary, including estimated variance components  
summary(random_model)
```

```
## Linear mixed model fit by REML ['lmerMod']  
## Formula: yield ~ 1 + (1 | variety)  
##  
## REML criterion at convergence: 21.6  
##  
## Scaled residuals:  
##      Min       1Q   Median       3Q      Max   
## -1.8839 -0.6181  0.1118  0.4962  2.7828   
##  
## Random effects:  
##   Groups   Name      Variance Std.Dev.  
##  variety (Intercept) 0.40693  0.6379  
## Residual              0.08374  0.2894  
## Number of obs: 30, groups:  variety, 3  
##  
## Fixed effects:  
##              Estimate Std. Error t value  
## (Intercept)   6.2100    0.3721   16.69
```

The random effects model treats the variety effects as random deviations from the overall mean. The term `(1 | variety)` means that each variety receives its own random intercept.

This model estimates two sources of variation:

- variation among soybean varieties,  $\sigma_{\alpha}^2$ ;
- residual plot-to-plot variation within varieties,  $\sigma^2$ .

The random effects model is appropriate when the selected varieties represent a sample from a broader population, and the goal is to estimate how much yield varies across soybean varieties in general.

## Confidence Intervals for Variance Components and the Overall Mean

Next, we construct confidence intervals for the random-effects variance component  $\sigma_\alpha^2$ , the residual variance  $\sigma^2$ , and the overall mean  $\mu$ .

```
# Compute confidence intervals using profile likelihood
# For lmer models, confint() can provide CIs for:
#   - fixed effects, such as the overall intercept mu
#   - variance components, such as the variety-level variance
#   - residual standard deviation
CIs <- confint(random_model, oldNames = FALSE)
```

```
## Computing profile confidence intervals ...
```

```
# Display the confidence intervals
CIs
```

```
##                2.5 %    97.5 %
## sd_(Intercept)|variety 0.2637525 1.5512218
## sigma                 0.2265053 0.3877781
## (Intercept)          5.3618584 7.0581407
```

The confidence intervals provide uncertainty quantification for the model parameters. In particular, the interval for the variety-level variance component reflects how much soybean yields vary across the broader population of soybean varieties.

## RCBD: Fixed vs. Random Block

In a randomized complete block design (RCBD), treatments are typically the main factor of interest. Blocks are included to account for nuisance variation.

Depending on the study design and inferential goal, blocks may be treated as either fixed or random:

- Treat blocks as fixed effects if the specific blocks in the study are the only blocks of interest.
- Treat blocks as random effects if the blocks are viewed as a random sample from a larger population of possible blocks.

## Load R Libraries

```
# lsmeans is used for estimated marginal means and pairwise comparisons
# lmerTest extends lme4 by providing approximate tests for fixed effects
library(lsmeans)
library(lmerTest)
```

## Read the data

```

# Response variable
x <- c(52, 47, 44, 51, 42, 60, 55, 49, 52,
      43, 56, 48, 45, 44, 38)

# Treatment labels: three treatments, each observed in five blocks
trt <- rep(c("A", "B", "C"), each = 5)

# Block labels: five blocks, repeated once for each treatment
blk <- rep(1:5, 3)

# Create the data frame
dat <- data.frame(x = x,
  trt = factor(trt), blk = factor(blk))

# Display the data
dat

```

```

##      x trt blk
## 1  52   A   1
## 2  47   A   2
## 3  44   A   3
## 4  51   A   4
## 5  42   A   5
## 6  60   B   1
## 7  55   B   2
## 8  49   B   3
## 9  52   B   4
## 10 43   B   5
## 11 56   C   1
## 12 48   C   2
## 13 45   C   3
## 14 44   C   4
## 15 38   C   5

```

## Fixed Block Model

```

# Fit an RCBD model treating both treatment and block as fixed effects
# This model estimates a separate mean adjustment for each block.
fixed_block_model <- lm(x ~ trt + blk, data = dat)

# ANOVA table for the fixed block model
anova(fixed_block_model)

```

```

## Analysis of Variance Table
##
## Response: x
##      Df Sum Sq Mean Sq F value    Pr(>F)
## trt    2   89.2    44.60   7.6239 0.0140226 *
## blk    4  363.6    90.90  15.5385 0.0007684 ***
## Residuals 8   46.8     5.85
## ---

```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

In the fixed block model, block effects are treated as specific parameters. This is appropriate when the five blocks are the exact blocks of interest.

The treatment  $F$ -test is based on the residual mean square after accounting for block-to-block variation.

## Random Block Model

```
# Fit an RCBD model treating treatment as fixed and block as random
# The term (1 | blk) allows each block to have its own random intercept.
random_block_model <- lmer(x ~ trt + (1 | blk), REML = TRUE,
                           data = dat)

# Model summary, including treatment effects and block-level variance component
summary(random_block_model)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: x ~ trt + (1 | blk)
##   Data: dat
##
## REML criterion at convergence: 71.1
##
## Scaled residuals:
##   Min       1Q   Median       3Q      Max
## -1.1417 -0.6147 -0.1494  0.5772  1.3390
##
## Random effects:
##   Groups   Name                Variance Std.Dev.
##   blk      (Intercept) 28.35      5.324
##   Residual                    5.85      2.419
## Number of obs: 15, groups: blk, 5
##
## Fixed effects:
##              Estimate Std. Error    df t value Pr(>|t|)
## (Intercept)   47.200     2.615   5.054  18.047 8.76e-06 ***
## trtB           4.600     1.530   8.000   3.007  0.0169 *
## trtC          -1.000     1.530   8.000  -0.654  0.5316
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##      (Intr) trtB
## trtB -0.292
## trtC -0.292  0.500
```

In this model, treatment is still treated as a fixed effect because the treatments A, B, and C are the main levels of interest.

The block effect is treated as random:

$$\beta_j \stackrel{i.i.d}{\sim} N(0, \sigma_\beta^2),$$

where  $\sigma_\beta^2$  measures the amount of variability among blocks.

## Pairwise Treatment Comparisons

```
# Pairwise comparisons among treatment means
# adjust = "none" reports unadjusted pairwise comparisons
lsmeans(random_block_model, list(pairwise ~ trt), adjust = "none")

## $'lsmeans of trt'
##   trt lsmean   SE    df lower.CL upper.CL
##   A     47.2 2.62 5.05     40.5     53.9
##   B     51.8 2.62 5.05     45.1     58.5
##   C     46.2 2.62 5.05     39.5     52.9
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
##
## $'pairwise differences of trt'
##   1      estimate    SE df t.ratio p.value
## A - B      -4.6 1.53  8   -3.007  0.0169
## A - C       1.0 1.53  8    0.654  0.5316
## B - C       5.6 1.53  8    3.661  0.0064
##
## Degrees-of-freedom method: kenward-roger
```

The pairwise comparisons compare the treatment means after accounting for block-to-block variability. These comparisons answer whether the treatments differ in their expected response, while treating blocks as a random source of variation.

## Computer Experiments

Computer experiments are commonly used when the response is generated by a deterministic simulator rather than a physical experiment. In this setting, careful design of input locations is important because each simulator run may be computationally expensive.

### Design: Latin Hypercube Design

A Latin Hypercube Design (LHD) is a space-filling design. It ensures that each input variable is sampled evenly across its range. This is especially useful for computer experiments because it gives broad coverage of the input space with relatively few runs.

This example compares a simple random design with a Latin Hypercube Design for two input variables.

```
library(lhs)

# Generate a 30-run, 2-variable maximin Latin Hypercube Design
# n = 30 gives the number of design points
# k = 2 gives the number of input variables
# dup controls how many candidate points are considered during optimization
LHD <- maximinLHS(n = 30, k = 2, dup = 5)
```

```
# Display the design matrix
LHD
```

```
##           [,1]      [,2]
## [1,] 0.866264525 0.1387696331
## [2,] 0.167571293 0.8891612049
## [3,] 0.647517735 0.2974568920
## [4,] 0.348647062 0.1002954967
## [5,] 0.758084917 0.5064742043
## [6,] 0.571996137 0.7832289367
## [7,] 0.378848546 0.6714900065
## [8,] 0.916249699 0.4375082639
## [9,] 0.164015017 0.5758015946
## [10,] 0.491146927 0.3539149375
## [11,] 0.461910762 0.0004080562
## [12,] 0.315570181 0.2342694231
## [13,] 0.681310908 0.6344067223
## [14,] 0.128712952 0.3049706984
## [15,] 0.773448692 0.8161208962
## [16,] 0.057733255 0.7409332994
## [17,] 0.429608661 0.8373715414
## [18,] 0.801326297 0.0471419359
## [19,] 0.727104778 0.3807668753
## [20,] 0.293117417 0.4112809720
## [21,] 0.251480547 0.7143090706
## [22,] 0.546274752 0.1714466918
## [23,] 0.878590383 0.6062267708
## [24,] 0.623696120 0.9035429719
## [25,] 0.003517932 0.4936997886
## [26,] 0.527844296 0.5615416433
## [27,] 0.963893820 0.2075926259
## [28,] 0.077968275 0.9560349269
## [29,] 0.979196194 0.9858824973
## [30,] 0.213459608 0.0696306478
```

The object LHD is a  $30 \times 2$  matrix. Each row represents one simulator run, and each column represents one input variable.

The argument `dup` is a tuning parameter used in the maximin optimization. Larger values may improve the space-filling quality of the design, but they also require more computation.

### Compare Random Design and Latin Hypercube Design

```
# Generate a simple random design for comparison
set.seed(123)
random_design <- cbind(runif(n = 30), runif(n = 30))

# Set graphical parameters
par(las = 1,
    mar = c(3.5, 3.5, 1.2, 0.5),
    mgp = c(2.2, 1, 0), mfrow = c(1, 2))

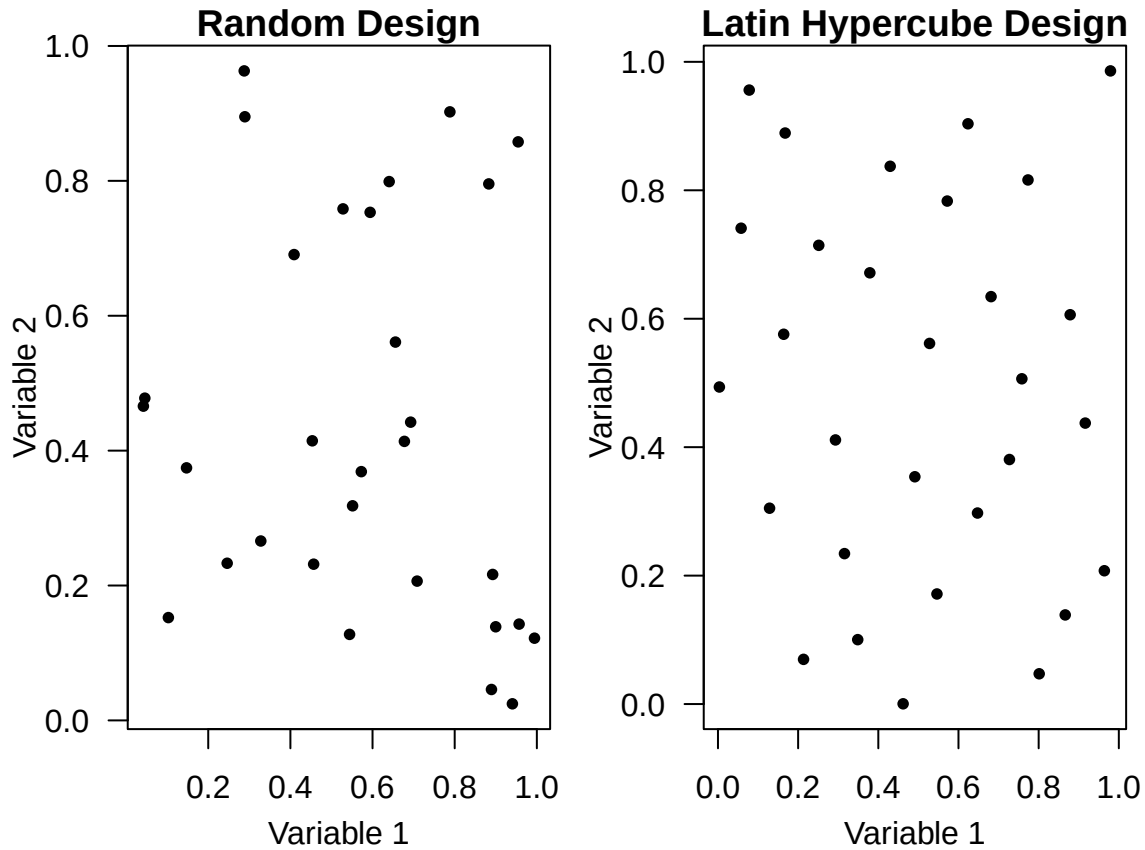
# Plot the simple random design
plot(random_design, cex = 0.8, pch = 16,
```

```

xlab = "Variable 1", ylab = "Variable 2",
main = "Random Design")

# Plot the Latin Hypercube Design
plot(LHD, cex = 0.8, pch = 16,
xlab = "Variable 1", ylab = "Variable 2",
main = "Latin Hypercube Design")

```



The random design may leave gaps or clusters in the input space. In contrast, the Latin Hypercube Design provides more even marginal coverage for each input variable.

### Analysis: Gaussian Process Model

After selecting input locations and running the computer experiment, we can use a Gaussian Process (GP) model as a statistical surrogate model.

A GP surrogate model is useful because it can:

- approximate the simulator response surface;
- predict responses at untried input settings;
- provide prediction uncertainty;
- support optimization and sensitivity analysis.

1. Load the data:

The following data set contains output from a computer experiment involving two input variables:

- g\_NaF
- g\_KDR

The third column is the simulator response.

```
# Load the neuron computer experiment data
neuron <- read.table(
  "http://deanvosdraguljic.ietsandbox.net/DeanVossDraguljic/R-data/neuron.txt",
  header = TRUE)

# Preview the first 10 rows
head(neuron, 10)
```

```
##           gNaFsc      gKdrsc fr
## 1  0.38593729 0.2120652 33
## 2  0.04666927 0.4594742  0
## 3  1.00000000 0.4473344 46
## 4  0.95467637 0.3351407 44
## 5  0.53334929 0.7981310 41
## 6  0.59166751 0.6042714 41
## 7  0.18570301 0.3799469 31
## 8  0.49927784 0.2444170 36
## 9  0.74609113 0.3949591 42
## 10 0.07269414 1.0000000  0
```

## 2. Fit a Gaussian Process Model

```
library(mlegp)

# Fit a Gaussian Process model
# The first two columns are input variables.
# The third column is the simulator response.
GP_fit <- mlegp(X = neuron[, 1:2], Z = neuron[, 3])

## no reps detected - nugget will not be estimated
##
## ===== FITTING GP # 1 =====
## running simplex # 1...
## ...done
## ...simplex #1 complete, loglike = -104.446501 (convergence)
## running simplex # 2...
## ...done
## ...simplex #2 complete, loglike = -104.446501 (convergence)
## running simplex # 3...
## ...done
## ...simplex #3 complete, loglike = -104.446502 (convergence)
## running simplex # 4...
## ...done
## ...simplex #4 complete, loglike = -104.446501 (convergence)
```

```
## running simplex # 5...
## ...done
## ...simplex #5 complete, loglike = -104.446501 (convergence)
##
## using L-BFGS method from simplex #1...
## iteration: 1,loglike = -104.446501
## ...L-BFGS method complete
##
## Maximum likelihood estimates found, log like = -104.446501
## creating gp object.....done
```

```
# Display a summary of the fitted GP model
summary(GP_fit)
```

```
##
## Total observations = 30
## Dimensions = 2
##
## mu = 27.61156
## sig2: 251.8769
## nugget: 0
##
## Correlation parameters:
##
##          beta a
## 1  5.027973 2
## 2 50.227290 2
##
## Log likelihood = -104.4465
##
## CV RMSE: 7.312656
## CV RMaxSE: 1020.783
```

The fitted GP model estimates a smooth response surface over the two-dimensional input space. Because this is a computer experiment, the GP is used as an emulator of the original simulator.

### 3. Make Predictions on a Grid

- Creates a grid of input values over the range  $[0, 1]$  for both variables.
- Uses the fitted GP model to predict responses and their standard errors on the grid.

```
# Create a prediction grid over the input space [0, 1] x [0, 1]
prediction_grid <- expand.grid(
  g_NaF = seq(0, 1, 0.02),
  g_KDR = seq(0, 1, 0.02))

# Predict the response and its standard error at each grid point
GP_pred <- predict(GP_fit,
  prediction_grid,
  se.fit = TRUE)
```

The prediction grid allows us to visualize the estimated response surface across the full input region. The standard errors quantify prediction uncertainty at each location in the input space.

#### 4. Visualize GP Predictions and Prediction Uncertainty

```
library(fields)

# Store grid values for plotting
grid_values <- seq(0, 1, 0.02)

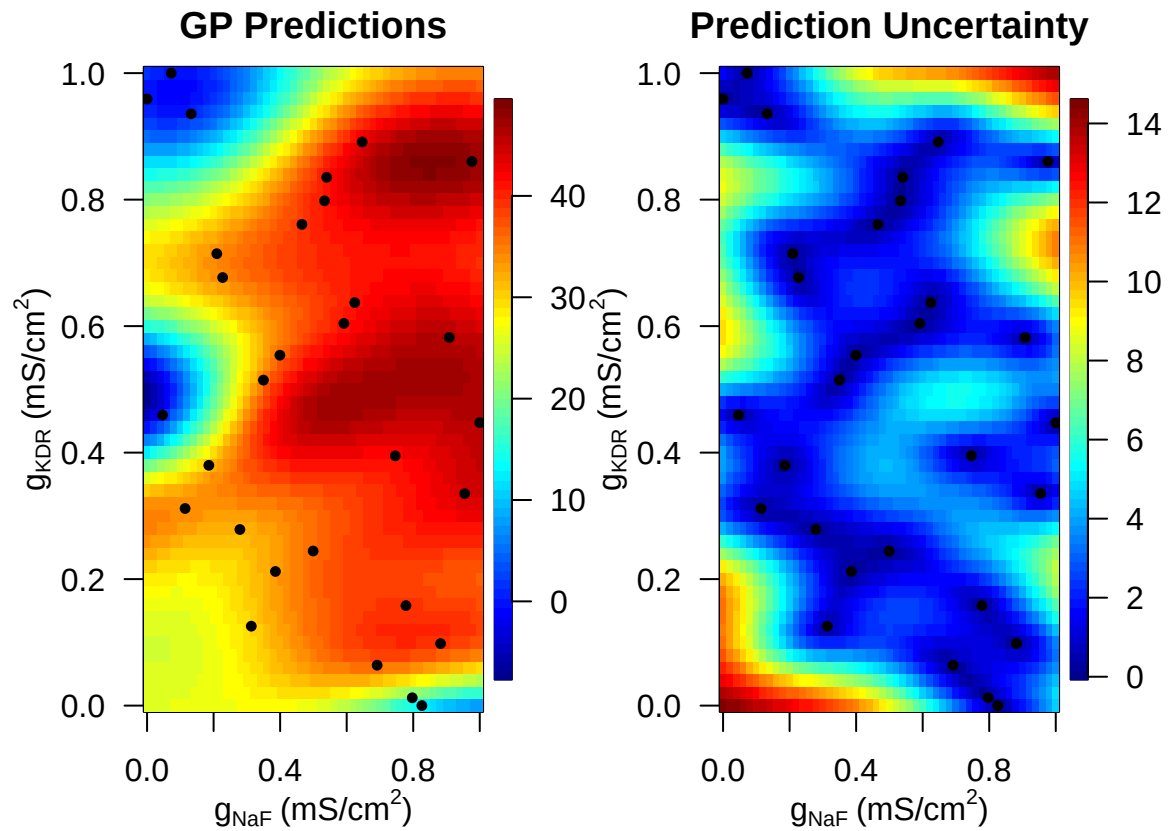
# Set graphical parameters
par(mfrow = c(1, 2), las = 1,
    mar = c(3.5, 3.5, 2.2, 0.8),
    mgp = c(2.2, 1, 0))

# Plot the GP predicted response surface
image.plot(grid_values, grid_values,
           matrix(GP_pred$fit, 51, 51),
           xlab = expression(g[NaF] ~ "(mS/cm" ^ 2 * ")" ),
           ylab = expression(g[KDR] ~ "(mS/cm" ^ 2 * ")" ),
           main = "GP Predictions")

# Add observed simulator runs
points(neuron[, 1:2], pch = 16, cex = 0.75)

# Plot the GP prediction standard errors
image.plot(grid_values, grid_values,
           matrix(GP_pred$se.fit, 51, 51),
           xlab = expression(g[NaF] ~ "(mS/cm" ^ 2 * ")" ),
           ylab = expression(g[KDR] ~ "(mS/cm" ^ 2 * ")" ),
           main = "Prediction Uncertainty")

# Add observed simulator runs
points(neuron[, 1:2], pch = 16, cex = 0.75)
```



The first plot shows the GP-predicted response surface. The second plot shows the prediction uncertainty. Prediction uncertainty is usually smaller near observed simulator runs and larger in sparsely sampled regions of the input space. This is one reason why space-filling designs, such as Latin Hypercube Designs, are useful for computer experiments.